Atividade P2 A.D Capítulo 6

Aluno: Kaynan Santos Freitas **Matricula:** 17529047

Preparação do ambiente

In [1]:
```
%pip install line_solver --quiet
```

```
Note: you may need to restart the kernel to use updated packages.
```

In [2]:
```
from itertools import product
from time import perf_counter

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

pd.set_option('display.precision', 5)
plt.rcParams['figure.figsize'] = (8, 4.5)
plt.rcParams['axes.grid'] = True
```

Questão 1

O sistema possui duas classes fechadas: consultas (Q) e atualizações (U). Primeiro usamos as populações originais $N_Q = 3$ e $N_U = 1$. Depois triplicamos a população de atualizações, obtendo $N_U = 3$.

In [3]:
```
recursos = ['CPU', 'Disco 1', 'Disco 2']
classes = ['Q', 'U']
D = np.array([
    [0.105, 0.375],
    [0.180, 0.480],
    [0.000, 0.240]
])

def resumo(D, N, Q, X, R):
    U = D * X
    por_classe = pd.DataFrame({
        'Classe': classes,
        'População': N,
        'Vazão': X,
        'Tempo de resposta': R.sum(axis=0)
    })
    por_recurso = pd.DataFrame({
        'Recurso': recursos,
        'Ocupação média': Q.sum(axis=1),
        'Utilização': U.sum(axis=1)
    })
    ocupacao = pd.DataFrame(Q, index=recursos, columns=classes)
    return por_classe, por_recurso, ocupacao
```

MVA exato

O método exato percorre todos os estados populacionais. Para cada classe, aplica o teorema das chegadas ao estado que possui um cliente a menos daquela classe.

In [4]:
```python
def mva_exato(D, N):
    N = np.array(N, dtype=int)
    m, c = D.shape
    ocupacoes = {tuple(np.zeros(c, dtype=int)): np.zeros((m, c))}
    estados = 0

    for estado in product(*[range(n + 1) for n in N]):
        if sum(estado) == 0:
            continue
        Q = np.zeros((m, c))
        for r in range(c):
            if estado[r] == 0:
                continue
            anterior = list(estado)
            anterior[r] -= 1
            Q_anterior = ocupacoes[tuple(anterior)]
            R_r = D[:, r] * (1 + Q_anterior.sum(axis=1))
            X_r = estado[r] / R_r.sum()
            Q[:, r] = X_r * R_r
        ocupacoes[estado] = Q
        estados += 1

    Q = ocupacoes[tuple(N)]
    R = np.zeros((m, c))
    X = np.zeros(c)
    for r in range(c):
        anterior = N.copy()
        anterior[r] -= 1
        R[:, r] = D[:, r] * (1 + ocupacoes[tuple(anterior)].sum(axis=1))
        X[r] = N[r] / R[:, r].sum()
    return Q, X, R, estados
```

MVA aproximado

A aproximação estima o estado com um cliente a menos a partir das ocupações do estado completo. O processo termina quando a maior diferença entre duas estimativas sucessivas de $n_{i,r}$ é menor ou igual a 0,001.

In [5]:
```python
def mva_aproximado(D, N, tolerancia=0.001, limite=10000):
    N = np.array(N, dtype=float)
    Q = D / D.sum(axis=0) * N

    for iteracao in range(1, limite + 1):
        R = np.zeros_like(D)
        X = np.zeros(len(N))
        novo_Q = np.zeros_like(D)

        for r in range(len(N)):
            fila_anterior = Q.sum(axis=1) - Q[:, r] / N[r]
            R[:, r] = D[:, r] * (1 + fila_anterior)
            X[r] = N[r] / R[:, r].sum()
            novo_Q[:, r] = X[r] * R[:, r]
```

```
        diferenca = np.max(np.abs(novo_Q - Q))
        Q = novo_Q
        if diferenca <= tolerancia:
            return Q, X, R, iteracao, diferenca

    raise RuntimeError('O método aproximado não convergiu')
```

In [6]:
```
def comparar_metodos(D, N):
    Qe, Xe, Re, estados = mva_exato(D, N)
    Qa, Xa, Ra, iteracoes, diferenca = mva_aproximado(D, N)
    tabela = pd.DataFrame({
        'Classe': classes,
        'X exato': Xe,
        'X aproximado': Xa,
        'Erro X (%)': 100 * np.abs(Xa - Xe) / Xe,
        'R exato': Re.sum(axis=0),
        'R aproximado': Ra.sum(axis=0),
        'Erro R (%)': 100 * np.abs(Ra.sum(axis=0) - Re.sum(axis=0)) /
Re.sum(axis=0)
    })
    esforco = pd.DataFrame({
        'Método': ['Exato', 'Aproximado'],
        'Estados ou iterações': [estados, iteracoes]
    })
    return tabela, esforco, (Qe, Xe, Re), (Qa, Xa, Ra), diferenca
```

In [7]:
```
comparacao_original, esforco_original, exato_original, aproximado_original,
diferenca_original = comparar_metodos(D, [3, 1])
comparacao_triplicada, esforco_triplicado, exato_triplicado,
aproximado_triplicado, diferenca_triplicada = comparar_metodos(D, [3, 3])

print('Cenário original: N = (3, 1)')
display(comparacao_original)
display(esforco_original)
print(f'Diferença final: {diferenca_original:.6f}')

print('Cenário com atualizações triplicadas: N = (3, 3)')
display(comparacao_triplicada)
display(esforco_triplicado)
print(f'Diferença final: {diferenca_triplicada:.6f}')
```

```
Cenário original: N = (3, 1)
```

	Classe	X exato	X aproximado	Erro X (%)	R exato	R aproximado	Erro R (%)
0	Q	4.09389	3.99688	2.36962	0.7328	0.75059	2.42713
1	U	0.40933	0.40692	0.58871	2.4430	2.45747	0.59220

	Método	Estados ou iterações
0	Exato	7
1	Aproximado	9

```
Diferença final: 0.000716
Cenário com atualizações triplicadas: N = (3, 3)
```

	Classe	X exato	X aproximado	Erro X (%)	R exato	R aproximado	Erro R (%)
0	Q	2.93810	2.84614	3.12966	1.02107	1.05406	3.23077
1	U	0.90645	0.89889	0.83337	3.30963	3.33744	0.84037

	Método	Estados ou iterações
0	Exato	15
1	Aproximado	12

```
Diferença final: 0.000671
```

In [8]:
```python
def medir_tempo(funcao, repeticoes=1000):
    inicio = perf_counter()
    for _ in range(repeticoes):
        funcao()
    return 1000 * (perf_counter() - inicio) / repeticoes

tempos = pd.DataFrame({
    'Cenário': ['Original', 'Original', 'Atualizações triplicadas',
'Atualizações triplicadas'],
    'Método': ['Exato', 'Aproximado', 'Exato', 'Aproximado'],
    'Tempo médio (ms)': [
        medir_tempo(lambda: mva_exato(D, [3, 1])),
        medir_tempo(lambda: mva_aproximado(D, [3, 1])),
        medir_tempo(lambda: mva_exato(D, [3, 3])),
        medir_tempo(lambda: mva_aproximado(D, [3, 3]))
    ]
})
tempos
```

Out[8]:

	Cenário	Método	Tempo médio (ms)
0	Original	Exato	0.14511
1	Original	Aproximado	0.34440
2	Atualizações triplicadas	Exato	0.26054
3	Atualizações triplicadas	Aproximado	0.47445

O MVA exato avalia todas as combinações das populações das classes, enquanto o aproximado trabalha apenas com o estado final e repete os cálculos até atingir a tolerância. Neste problema a medição de tempo mostra que o método exato ainda é mais rápido. A vantagem do aproximado é assintótica: o número de estados do exato cresce como $\prod_r (N_r + 1)$, enquanto o aproximado mantém somente o estado final e tende a ser vantajoso para populações e números de classes maiores.

Questão 2

No modelo atual, consultas usam 0,180 s do Disco 1 e não usam o Disco 2. No cenário balanceado, essa demanda é dividida igualmente: 0,090 s em cada disco.

In [9]:
```
D_balanceado = D.copy()
D_balanceado[1, 0] = 0.090
D_balanceado[2, 0] = 0.090

def resultado_cenario(nome, D_cenario):
    Q, X, R, estados = mva_exato(D_cenario, [3, 1])
    _, recursos_cenario, _ = resumo(D_cenario, [3, 1], Q, X, R)
    return pd.DataFrame({
        'Cenário': nome,
        'Classe': classes,
        'Vazão': X,
        'Tempo de resposta': R.sum(axis=0)
    }), recursos_cenario

classes_atual, recursos_atual = resultado_cenario('Atual', D)
classes_balanceado, recursos_balanceado = resultado_cenario('Balanceado',
D_balanceado)

display(pd.concat([classes_atual, classes_balanceado], ignore_index=True))

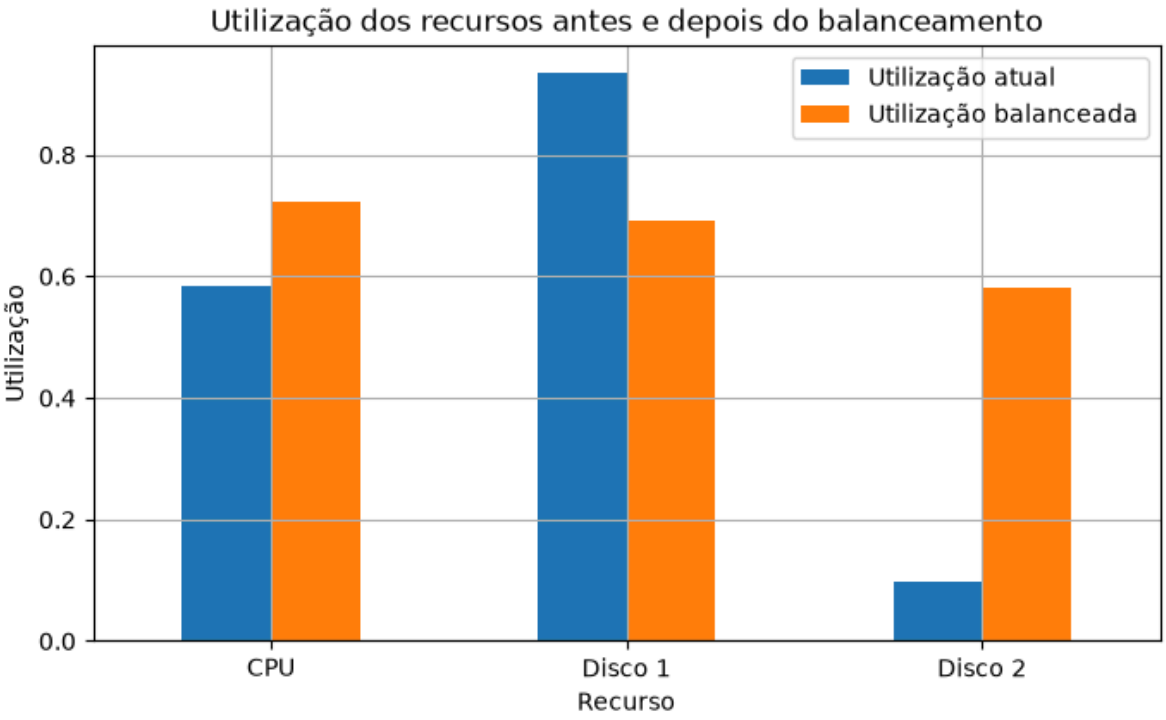
comparacao_recursos = recursos_atual.merge(recursos_balanceado, on='Recurso',
suffixes=(' atual', ' balanceada'))
comparacao_recursos
```

	Cenário	Classe	Vazão	Tempo de resposta
0	Atual	Q	4.09389	0.73280
1	Atual	U	0.40933	2.44300
2	Balanceado	Q	5.24653	0.57181
3	Balanceado	U	0.45612	2.19241

Out[9]:

	Recurso	Ocupação média atual	Utilização atual	Ocupação média balanceada	Utilização balanceada
0	CPU	1.12162	0.58336	1.5377	0.72193
1	Disco 1	2.78014	0.93338	1.4105	0.69112
2	Disco 2	0.09824	0.09824	1.0518	0.58166

In [10]:
```
grafico = comparacao_recursos.set_index('Recurso')[['Utilização atual',
'Utilização balanceada']]
grafico.plot(kind='bar', rot=0)
plt.ylabel('Utilização')
plt.title('Utilização dos recursos antes e depois do balanceamento')
plt.show()
```



O balanceamento reduz a utilização do Disco 1 e aproveita a capacidade disponível do Disco 2. Para Q, o tempo de resposta cai de aproximadamente 0,733 s para 0,572 s (melhoria de 22%) e a vazão cresce cerca de 28%. Para U, o tempo cai de 2,443 s para 2,192 s (melhoria de 10%) e a vazão cresce aproximadamente 11,4%.

Questão 3

As classes Q e U são abertas, com taxas de chegada conhecidas e limites de multiprogramação $N_Q^{max} = 5$ e $N_U^{max} = 3$. A classe I é interativa, com $M_I = 50$ terminais, tempo médio de pensar $Z_I = 15$ s e no máximo $N_I^{max} = 5$ requisições ativas.

Para reproduzir o modelo do QSolver/1, resolvemos a rede mista por decomposição. Enquanto houver menos de cinco requisições interativas ativas, vale a lei interativa $X_I = M_I/(Z_I + R_I)$. Quando esse limite é alcançado, somente cinco requisições disputam os dispositivos; as demais aguardam admissão, e seu tempo também faz parte da resposta percebida pelo usuário.

In [11]:
```
D3 = np.array([
    [0.060, 0.100, 0.090],  # CPU
    [0.030, 0.030, 0.045],  # D1
    [0.060, 0.090, 0.000]   # D2
])
classes3 = ['Q', 'U', 'I']
NMAX = {'Q': 5, 'U': 3, 'I': 5}
Z_I = 15
```

In [12]:
```
def mva_classe_fechada(demandas, populacao, think_time=0.0):
    Q = np.zeros(len(demandas))
    for n in range(1, populacao + 1):
        R = demandas * (1 + Q)
        X = n / (think_time + R.sum())
        Q = X * R
```

```
    return Q, X, R


def resolver_misto(lambda_q=3.0, terminais=50, fator_cpu=1.0, fator_d1=1.0):
    demandas = D3.copy()
    demandas[0, :] /= fator_cpu
    demandas[1, :] /= fator_d1

    lambdas = np.array([lambda_q, 1.5])
    utilizacao_aberta = demandas[:, :2] @ lambdas
    if np.any(utilizacao_aberta >= 1):
        raise ValueError('A sub-rede aberta é instável')

    demandas_alongadas = demandas[:, 2] / (1 - utilizacao_aberta)

    # Primeiro resolvemos sem o limite para verificar quantas requisições
interativas ficariam ativas na sub-rede dos dispositivos.
    Q_livre, X_livre, R_livre = mva_classe_fechada(
        demandas_alongadas, terminais, think_time=Z_I
    )

    if Q_livre.sum() <= NMAX['I'] + 1e-12:
        Q_fechada, X_i = Q_livre, X_livre
        resposta_i = R_livre.sum()
        limite_ativo = False
    else:
        # Com o limite ativo, cinco requisições circulam nos dispositivos.
        Q_fechada, X_i, _ = mva_classe_fechada(
            demandas_alongadas, NMAX['I'], think_time=0
        )
        resposta_i = terminais / X_i - Z_I
        limite_ativo = True

    R_abertas = demandas[:, :2] * (1 + Q_fechada[:, None]) / (1 -
utilizacao_aberta[:, None])
    respostas_abertas = R_abertas.sum(axis=0)
    populacoes_abertas = lambdas * respostas_abertas
    if np.any(populacoes_abertas > np.array([NMAX['Q'], NMAX['U']]) + 1e-12):
        raise ValueError('Um limite de multiprogramação de classe aberta foi
excedido')

    respostas = np.array([*respostas_abertas, resposta_i])
    vazoes = np.array([lambda_q, 1.5, X_i])
    utilizacao_total = utilizacao_aberta + X_i * demandas[:, 2]

    return {
        'respostas': pd.DataFrame({'Classe': classes3, 'Tempo de resposta':
respostas}),
        'vazoes': pd.DataFrame({'Classe': classes3, 'Vazão': vazoes}),
        'utilizacoes': pd.DataFrame({'Recurso': recursos, 'Utilização':
utilizacao_total}),
        'limite_interativo_ativo': limite_ativo,
    }
```

In [13]:
```
def tempos_resposta(resultado):
    return resultado['respostas'].copy()


def utilizacoes(resultado):
    return resultado['utilizacoes'].copy()
```

(a) Tempos de resposta no cenário original

In [14]:
```
base = resolver_misto()
display(tempos_resposta(base))
display(base['vazoes'])
display(utilizacoes(base))
```

	Classe	Tempo de resposta
0	Q	0.28719
1	U	0.43621
2	I	0.29566

	Classe	Vazão
0	Q	3.0000
1	U	1.5000
2	I	3.2689

	Recurso	Utilização
0	CPU	0.6242
1	Disco 1	0.2821
2	Disco 2	0.3150

(b) Aumento de 95% na chegada das consultas

A nova taxa é $3,0 \times 1,95 = 5,85$ transações por segundo.

In [15]:
```
aumento = resolver_misto(lambda_q=5.85)
comparacao_b = tempos_resposta(base).merge(
    tempos_resposta(aumento), on='Classe', suffixes=(' original', ' com
aumento')
)
comparacao_b['Variação (%)'] = 100 * (
    comparacao_b['Tempo de resposta com aumento'] /
    comparacao_b['Tempo de resposta original'] - 1
)
display(comparacao_b)
display(utilizacoes(aumento))
```

	Classe	Tempo de resposta original	Tempo de resposta com aumento	Variação (%)
0	Q	0.28719	0.44263	54.12653
1	U	0.43621	0.68676	57.43700
2	I	0.29566	0.47839	61.80393

	Recurso	Utilização
0	CPU	0.79173
1	Disco 1	0.36586
2	Disco 2	0.48600

(c) Comparação dos upgrades

Um recurso duas vezes mais rápido possui metade da demanda de serviço.

In [16]:
```
cenarios = {
    'Sem upgrade': resolver_misto(lambda_q=5.85),
    'D1 duas vezes mais rápido': resolver_misto(lambda_q=5.85, fator_d1=2.0),
    'CPU duas vezes mais rápida': resolver_misto(lambda_q=5.85, fator_cpu=2.0)
}

resultado_c = []
for nome, resultado in cenarios.items():
    parte = tempos_resposta(resultado)
    parte.insert(0, 'Cenário', nome)
    resultado_c.append(parte)

comparacao_c = pd.concat(resultado_c, ignore_index=True)
display(comparacao_c)

print('A troca da CPU produz a maior redução dos tempos de resposta.')
```

	Cenário	Classe	Tempo de resposta
0	Sem upgrade	Q	0.44263
1	Sem upgrade	U	0.68676
2	Sem upgrade	I	0.47839
3	D1 duas vezes mais rápido	Q	0.41467
4	D1 duas vezes mais rápido	U	0.65943
5	D1 duas vezes mais rápido	I	0.43662
6	CPU duas vezes mais rápida	Q	0.21409
7	CPU duas vezes mais rápida	U	0.30570
8	CPU duas vezes mais rápida	I	0.14534

```
A troca da CPU produz a maior redução dos tempos de resposta.
```

(d) Número máximo de terminais

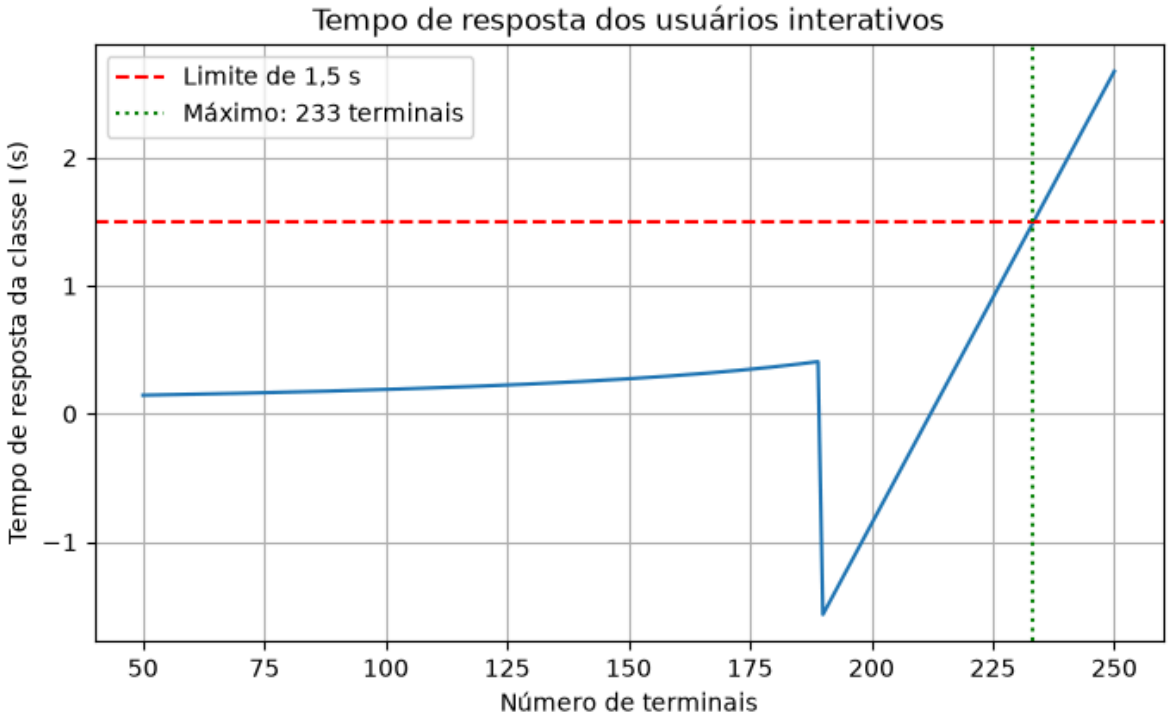
Mantemos a taxa de consultas aumentada e a CPU duas vezes mais rápida. Variamos M_I de 50 a 250, respeitando $Z_I = 15$ s e o limite $N_I^{max} = 5$. O requisito é manter o tempo total de resposta dos usuários interativos abaixo de 1,5 segundo.

In [17]:
```
linhas = []
for quantidade in range(50, 251):
    resultado = resolver_misto(lambda_q=5.85, terminais=quantidade,
fator_cpu=2.0)
    respostas = tempos_resposta(resultado).set_index('Classe')['Tempo de
resposta']
    linhas.append({
        'Terminais': quantidade,
        'Tempo de resposta I': respostas['I'],
        'Limite de multiprogramação ativo': resultado['limite_interativo_ativo']
    })

resultado_d = pd.DataFrame(linhas)
aceitaveis = resultado_d[resultado_d['Tempo de resposta I'] < 1.5]
maximo = int(aceitaveis['Terminais'].max()) if not aceitaveis.empty else None

plt.plot(resultado_d['Terminais'], resultado_d['Tempo de resposta I'])
plt.axhline(1.5, color='red', linestyle='--', label='Limite de 1,5 s')
plt.axvline(maximo, color='green', linestyle=':', label=f'Máximo: {maximo}
terminais')
plt.xlabel('Número de terminais')
plt.ylabel('Tempo de resposta da classe I (s)')
plt.title('Tempo de resposta dos usuários interativos')
plt.legend()
plt.show()

comparacao_limite = resultado_d[
    resultado_d['Terminais'].isin([maximo, maximo + 1])
] [['Terminais', 'Tempo de resposta I']].copy()
comparacao_limite['Requisito R < 1,5 s'] = np.where(
    comparacao_limite['Tempo de resposta I'] < 1.5, 'Atende', 'Não atende'
)
display(
    comparacao_limite.style
    .hide(axis='index')
    .format({'Tempo de resposta I': '{:.4f} s'})
)
print(f'Maior número de terminais com resposta abaixo de 1,5 s: {maximo}')
```



Terminais	Tempo de resposta I	Requisito R < 1,5 s
233	1.4726 s	Atende
234	1.5433 s	Não atende

```
Maior número de terminais com resposta abaixo de 1,5 s: 233
```

Conclusão

O MVA aproximado produz resultados próximos aos do método exato e torna-se computacionalmente vantajoso à medida que o espaço de estados cresce. O balanceamento de E/S reduz a concentração de carga no Disco 1. No modelo misto da questão 3, a CPU é o principal gargalo e sua substituição traz uma melhora muito maior que a troca do Disco 1. Considerando os 15 segundos de pensar e o limite de cinco requisições interativas ativas, a configuração com CPU duas vezes mais rápida suporta no máximo 233 terminais com tempo de resposta inferior a 1,5 segundo.